Dependency Injection və xidmət ömürləri

Dependency Injection, qısa adı DI, bir obyektin ehtiyac duyduğu başqa obyektləri kənardan almasıdır. Bu dərsdə “nə qeydiyyatdan keçirirəm?”, “obyekt nə vaxt yaranır?”, “kim paylaşır?” və “kim təmizləyir?” suallarını ayrı cavablandıracağıq. Əsas məqsəd sintaksisi əzbərləməkdən əlavə yanlış lifetime seçiminin səbəb olduğu xətanı qabaqcadan görməkdir.

İlkin materiallar: `DependencyInjection.pdf`, `di_practical.pdf`, `files/03_di_architecture.pdf`. Həmin sənədlərdəki scope, yaranma anı, DIP və xidmətlərin təmizlənməsi izahlarını burada dəqiqləşdiririk.

Mündəricat

Dependency Injection və xidmət ömürləri **1**

1 Asılılıq və abstraksiya nədir . . . 2
2 IoC DI və DIP fərqi . . . 2
3 Qeydiyyat resolve və ilk yaranma . . . 3
4 Üç lifetime və dəqiq sərhədlər . . . 3
5 Captive dependency niyə yaranır . . . 4
6 Fon işində ayrıca scope . . . 4
7 Təmizlənmənin sahibi kimdir . . . 5
8 Birdən çox implementasiya və factory . . . 5
9 Səhvləri necə tanımalı . . . 5
10 Müsahibə və məşq . . . 6
11 Rəsmi mənbələr . . . 6

1 Asılılıq və abstraksiya nədir

`OrderService` sifarişi saxlamalıdırsa və bu iş üçün `OrderStore` istifadə edirsə, store onun **dependency**, yəni asılılığıdır. **Sinif** məlumat və davranışı təsvir edən tipdir. **Obyekt** həmin tipdən yaradılmış nüsxədir. **Interface** sinfin hansı əməliyyatları təqdim etməli olduğunu göstərən müqavilədir. **Implementation** müqavilənin konkret necə yerinə yetirildiyidir.

Abstraksiya istifadəçiyə vacib davranışı göstərib daxili təfərrüatı gizlədən modeldir. Interface abstraksiyanın bir üsuludur, yeganə üsulu deyil. **Coupling** bir hissənin digər hissənin təfərrüatına nə qədər bağlı olmasıdır. Concrete SQL sinfinə birbaşa bağlı xidmət onu dəyişəndə daha çox təsirlənir. Müqaviləyə bağlı xidmət eyni davranış müqaviləsini qoruyan başqa implementasiya ilə işləyə bilər.

C#

```
public interface IOrderStore
{
    void Save(string order);
}

public sealed class OrderService
{
    private readonly IOrderStore _store;

    public OrderService(IOrderStore store)
    {
        _store = store;
    }

    public void Place(string order)
    {
        _store.Save(order);
    }
}
```

`public` üzvün kənardan görünməsini, `private` yalnız sinif daxilində görünməsini bildirir. `sealed` sinifdən miras almağı bağlayır. `readonly` həmin sahənin istinadının konstruktor xaricində dəyişdirilməsini qadağan edir; göstərdiyi obyektin daxilini avtomatik dəyişməz etmir. `void` nəticə dəyəri qaytarılmadığını, `string` mətn tipini bildirir. `Place` iş metodudur, `OrderService(...)` konstruktordur.

Bu sinif store-u özü yaratmır. `new OrderService(new MemoryOrderStore())` yazmaq da manual DI-dır. `MemoryOrderStore` müqaviləni yerinə yetirən, yaddaşda saxlayan nümunə sinif ola bilər. Container DI üçün məcburi deyil. Eyni zamanda DI mütləq interface tələb etmir; konkret sinif də xaricdən verilə bilər.

2 IoC DI və DIP fərqi

IoC, Inversion of Control, idarəetmənin çərçivəyə və ya başqa mexanizmə ötürülməsidir. Məsələn, HTTP gələndə hansı controller-i çağıracağını framework seçir. DI obyekt asılılıqlarının verilməsi sahəsində IoC tətbiqidir.

DIP, Dependency Inversion Principle, yüksək səviyyəli iş qaydalarının aşağı səviyyəli texniki detallara deyil, uyğun abstraksiyalara bağlı olmasını istəyən dizayn prinsipidir. “Controller yalnız service-i bilir” qat qaydası təkbaşına DIP deyil. Service birbaşa konkret SQL sinfinə bağlıdırsa qatlar olsa da DIP məqsədi tam əldə olunmaya bilər. DI isə həmin dizayndakı asılılığın xaricdən verilmə texnikasıdır. DI ilə zəif dizaynı da qurmaq mümkündür.

Composition root tətbiqin konkret implementasiyaları birləşdirdiyi mərkəzdir; .NET-də çox vaxt `Program.cs` və qeydiyyat funksiyalarıdır. Biznes metodlarının hər yerində container axtarmaq əvəzinə obyekt qrafı burada qurulur. Obyekt qrafı bir obyektin digər obyektlərdən ibarət asılılıq şəbəkəsidir.

3 Qeydiyyat resolve və ilk yaranma

`IServiceCollection` xidmət təsvirlərinin siyahısıdır. Qeydiyyat “bu tip istənilsə bunu yarat və bu ömürlə paylaş” qaydasını əlavə edir. **Resolve** həmin qaydaya əsasən xidmət nüsxəsini əldə etməkdir. `IServiceProvider` resolve edən obyektdir. `GetRequiredService<T>()` qeydiyyat yoxdursa xəta verir; `GetService<T>()` isə uyğun hallarda `null`, yəni obyektin olmaması dəyərini qaytarır.

C#

```
builder.Services.AddScoped<IOrderStore, SqlOrderStore>();
builder.Services.AddScoped<OrderService>();
```

Bu fraqmentdə `SqlOrderStore` ayrıca yazılmış implementasiyadır, `builder` tətbiq qurucusudur. `OrderService` tələb olunanda container konstruktorundakı `IOrderStore` üçün də resolve edir. **Generic** `<T>` yazılışı müxtəlif tiplərlə işləyən ümumi tip və ya metod mexanizmidir. `<IOrderStore, SqlOrderStore>` müqavilə ilə implementasiyanı bağlayır.

Container lazım olmayan bütün servisləri startup-da yaratmır. Singleton çox vaxt ilk resolve zamanı yaranır. `AddSingleton(new Cache())` kimi hazır nüsxə verilirsə nüsxəni proqramçı artıq yaradıb. Qeydiyyatın mövcudluğu ilə constructor-un işə düşməsi ayrı hadisələrdir.

4 Üç lifetime və dəqiq sərhədlər

Lifetime	Paylaşılma qaydası
Transient	Container hər resolve zamanı yeni nüsxə yaradır. Eyni nüsxəni dəyişəndə saxlayıb yenidən istifadə etsən yeni obyekt yaranmır.
Scoped	Eyni scope və eyni qeydiyyat daxilində eyni nüsxə qaytarılır. Başqa scope ayrıca nüsxə alır.
Singleton	Eyni root provider və qeydiyyat daxilində bir nüsxə paylaşılır. Bir neçə proses və ya ayrıca provider ayrıca nüsxə saxlayır.

İki resolve və iki scope düşün: S1 daxilində T1 və T2 transient nüsxələr, C1 və C1 scoped nüsxələr, G və G singleton nüsxələr alınır. S2 daxilində T3 və T4, C2 və C2, yenə G və G alınır. Beləcə transient üçün resolve sayı, scoped üçün scope sayı, singleton üçün provider sərhədi əsasdır.

State obyektin saxladığı dəyişə bilən məlumatdır. **Stateless** xidmət çağırışlar arasında istifadəçiyə aid vəziyyət saxlamır. Stateless olması xidməti avtomatik singleton etməyə məcbur etmir. **Thread safety** eyni obyektə paralel girişlərin düzgün davranışı pozmamasıdır. Singleton-da request-ə aid istifadəçi ID-si saxlamaq iki istifadəçinin məlumatını qarışdıra bilər. Sadəcə onu singleton qeydiyyata almaq thread safety yaratmır.

5 Captive dependency niyə yaranır

Captive dependency uzunömürlü obyektin daha qısa ömür üçün nəzərdə tutulan obyekti özündə saxlamasıdır. Singleton constructor-u scoped `DbContext` alıb sahədə saxlayarsa context sorğu bitəndə yox olmalı olduğu sərhəddən uzun istifadə edilər. Scope validation aktivdirsə bu səhv erkən aşkar edilir. Validation qaydaların yoxlanmasıdır.

C#

```
// Problemlidir: Cache uzun, db ise qisa omurludur.
public class Cache(AppDbContext db)
{
    private readonly AppDbContext _db = db;
}
```

Burada constructor-un qısa forması C# 12 primary constructor yazılışıdır. `AppDbContext` sənin DbContext-dən törəyən sinfindir. DbContext dəyişiklik izləməsi və verilənlər bazası əməliyyatları aparır, paralel əməliyyatlara uyğun deyil. Daha uzun ömür köhnə izlənən məlumatı, yaddaş artımını və eyni anda istifadə xətalarını da yarada bilər.

Singleton-un transient alması container baxımından mümkündür. Amma singleton transient-i bir dəfə alıb saxladıqda həmin konkret obyekt də singleton qədər istifadə oluna bilər. “Transient yazmışam, deməli hər metod çağırışında yenidir” nəticəsi yanlışdır.

6 Fon işində ayrıca scope

Aşağıdakı nümunədə singleton işçi yalnız scope fabriki saxlayır. Hər `CountAsync` çağırışı yeni scope açır, onun context-i ilə işi bitirir və scope-u bağlayır. Scope-dan alınmış context sahədə saxlanmır.

C#

```
public sealed class Counter(IServiceScopeFactory scopes)
{
    public async Task<int> CountAsync(CancellationToken ct)
    {
        await using var scope = scopes.CreateAsyncScope();
        var db = scope.ServiceProvider
            .GetRequiredService<AppDbContext>();

        return await db.Orders.CountAsync(ct);
    }
}
```

`IServiceScopeFactory` scope yaradan framework xidmətidir. `await using` scope-un asinxron təmizlənməsini gözləyir. `Task<int>` tamamlananda tam ədəd nəticəsi verən əməliyyatı təmsil edir. `CancellationToken` ləğv siqnalıdır. `CountAsync` SQL ilə say hesablayan EF Core metodudur. Bu fraqment `Microsoft.Extensions.DependencyInjection` və `Microsoft.EntityFrameworkCore` ad sahələrindən istifadə edir. Alternativ olaraq `IDbContextFactory<T>` ilə ayrıca context yaradıla bilər; həmin yaradılmış context-i istifadə edən kod dispose etməlidir.

Adi constructor əsasında middleware tətbiq üçün bir dəfə qurula bilər. Scoped xidməti orada constructor ilə tutmaq əvəzinə `InvokeAsync` parametrindən almaq və ya hər sorğu üçün aktivləşən `IMiddleware` modelini seçmək lazımdır. `InvokeAsync` middleware-in sorğunu işləyən metodudur.

7 Təmizlənmənin sahibi kimdir

Ownership, sahiblik, resursu kimin təmizləməli olduğunu bildirir. Container yaratdığı disposable transient və scoped obyektləri scope-un, singleton-u isə provider-in sonunda təmizləyir. Disposable `IDisposable` və ya `IAsyncDisposable` ilə resurs buraxa bilən obyektdir. Constructor-la verilmiş asılılığı onu alan hər sinif ayrıca dispose etsə başqa istifadəçi də işləməyən obyekt alar.

Transient disposable obyekti root provider-dən davamlı resolve etmək container bağlanana qədər çox nüsxənin saxlanmasına səbəb ola bilər. Buna görə transient-in qeydiyyat adı həmişə qısa real yaddaş ömrünə zəmanət vermir. Root provider əsas container-dir; ayrıca yaradılmış scope deyil.

`AddSingleton(existingObject)` ilə xaricdə yaradılmış nüsxə verilibsə onun təmizlənməsini proqramçı idarə edir. `Dispose` yalnız resurs buraxır, obyektin managed yaddaşını dərhal silmir. Əlçatmaz obyektin yaddaşını GC sonradan geri ala bilər.

8 Birdən çox implementasiya və factory

Eyni interface üçün bir neçə qeydiyyat varsa, birbaşa tək xidmət resolve edərkən built-in container adətən son qeydiyyatı qaytarır. `IEnumerable<T>` isə bütün uyğun qeydiyyatları qeydiyyat sırası ilə verir. IEnumerable elementləri bir-bir gəzməyə imkan verən müqavilədir.

Factory obyektin yaradılmasını və ya uyğun nüsxənin seçilməsini ayrı yerə toplayan üsuldur. Runtime, proqram işləyən vaxt, gələn “card” və ya “cash” seçiminə görə fərqli xidmət seçilə bilər. DI-da factory delegate, çoxsaylı qeydiyyat və keyed services kimi imkanlar var. “DI runtime seçim edə bilmir” iddiası düzgün deyil. Keyed service xidmətə tipdən əlavə açar ilə müraciət etməyə imkan verir; istifadə olunan .NET versiyasında dəstəyini yoxlamaq lazımdır.

Constructor injection məcburi asılılıqları obyekt yaradılarkən verir. **Method injection** asılılığı konkret metod çağırışında verir. **Property injection** asılılığı property, yəni obyektin xassəsi ilə verir; built-in container bunu ümumi avtomatik davranış kimi təqdim etmir. Məcburi asılılığı sonradan verilə bilən property-də saxlamaq yarımçıq obyekt riskidir.

9 Səhvləri necə tanımalı

Service Locator iş metodunun container-dən gizli şəkildə xidmət axtarmasıdır. Asılılıqlar constructor-da görünmür və test çətinləşir. Ancaq composition root və scope açan infrastruktur kodunda provider-dən resolve etmək normaldır. `IServiceProvider` sözünün görünməsi təkbaşına bütün hallarda pis dizayn demək deyil.

Circular dependency A-nın B-yə, B-nin də A-ya birbaşa və ya aralıq obyektlərlə ehtiyacı olmasıdır. Həll üçün məsuliyyətləri yenidən bölmək, ortaq davranışı çıxarmaq və ya qarşılıqlı çağırışı dəyişmək lazımdır. Mediator əlavə etmək hər dövrəni avtomatik aradan qaldırmır; işləmə vaxtında yenə A-B-A çağırış dövrəsi qala bilər.

Constructor-da şəbəkə və baza çağırışı obyekt yaradılmasını gözlənilmədən bahalı və xətaya açıq edir. Constructor ilkin vəziyyəti qursun, I/O, yəni giriş çıxış əməliyyatı, ayrıca gözlənilə bilən metodda aparılsın. Biznes kodunda `BuildServiceProvider()` çağırıb ikinci container yaratmaq da singleton-ların təkrarlanmasına və lifetime qarışıqlığına səbəb ola bilər.

10 Müsahibə və məşq

Qısa cavab. DI asılılığı xaricdən verməklə obyektin yaradılma məntiqini iş məntiqindən ayırır. Container bunu avtomatlaşdırır. Transient hər resolve-da, scoped hər scope-da yenidir; singleton provider daxilində paylaşılır. Seçim paylaşılmış vəziyyətə, thread safety-yə, resurs sahibinə və əməliyyat sərhədinə əsaslanır.

Test nəyi sübut edir? Fake store saxlanılan sifarişi yaddaşda qeyd edən sadə implementasiya ola bilər. Onu `OrderService`-ə verib `Place` çağırırsan və düzgün dəyərin store-a ötürüldüyünü yoxlayırsan. Fake sadələşdirilmiş işlək əvəzləyicidir, mock isə çağırış gözləntiləri yoxlanan əvəzləyicidir. Bu test real SQL-in işlədiyini sübut etmir; onun üçün baza ilə integration test, yəni bir neçə hissənin birlikdə yoxlanması lazımdır.

Məşq. Konstruktorunda təsadüfi `Guid` yaradan `Probe` sinfi yaz. Guid 128 bitlik identifikatordur. Eyni scope-da iki dəfə, ikinci scope-da bir dəfə resolve et. Scoped üçün ilk iki ID eyni, sonuncu fərqli olmalıdır. Singleton-da hamısı eyni, transient-də hamısı ayrı olacaq. Sonra transient-i singleton sahəsində saxla: onun ID-si hər metod çağırışında dəyişməyəcək. Bu nəticə lifetime ilə real istinad ömrünün fərqini göstərir.

11 Rəsmi mənbələr

- Microsoft Learn - Dependency injection overview
- Microsoft Learn - Service lifetimes
- Microsoft Learn - Dependency injection guidelines
- Microsoft Learn - Architectural principles

Yoxlama tarixi 28 oktyabr 2026. Kodlar mövzunu göstərən fraqmentlərdir; `AppDbContext`, `Orders` və SQL implementasiyası layihədə ayrıca qurulur.